

are instructions that allow the programmer to alter the contents of the register) the program can "build" an address in the address register prior to executing a *memory reference* instruction (i.e., an instruction that reads data from memory, writes data to memory or operates on data stored in memory).

### ARITHMETIC/LOGIC UNIT (ALU):

All processors contain an arithmetic/logic unit, which is often referred to simply as the ALU. By way of analogy, the ALU may be thought of as a super adding machine with its keys commanded automatically by the control signals developed in the instruction decoder and the control circuitry. This is essentially how the first stored-program digital computer was conceived.

The ALU naturally bears little resemblance to a desk-top adder. The major difference is that the ALU calculates by creating an electrical analogy, rather than by mechanical analogy. Another important difference is that the ALU uses binary techniques — rather than decimal methods — for representing and manipulating numbers. In principle, however, it is convenient to think of the ALU as an electronically controlled calculator.

The ALU must contain an *adder* which is capable of combining the contents of two registers in accordance with the logic of binary arithmetic. This provision permits the processor to perform arithmetic manipulations on the data it obtains from memory and from its other inputs.

Using only the basic adder a capable programmer can write routines which will subtract, multiply and divide, giving the machine complete arithmetic capabilities. In practice, however, most ALUs provide other built-in functions, including hardware subtraction, boolean logic operations, and shift capabilities.

The ALU contains *flag bits* which register certain conditions that arise in the course of arithmetic manipulations. Flags typically include *carry* and *zero*. It is possible to program jumps which are conditionally dependent on the status of one or more flags. Thus, for example, the program may be designed to jump to a special routine, if the carry bit is set following an addition instruction. The presence of a carry generally indicates an overflow in the accumulator, and sometimes calls for special processing actions.

### CONTROL CIRCUITRY:

The control circuitry is the primary functional unit within a CPU. Using clock inputs, the control circuitry maintains the proper sequence of events required for any processing task. After an instruction is fetched and decoded, the control circuitry issues the appropriate signals (to units both internal and external to the CPU) for initiating the proper processing action. Often the control circuitry will be capable of responding to external signals, such as an interrupt request. An *interrupt* request will cause the control circuitry to temporarily interrupt main program execution, jump to a special routine to service the interrupting device, then automatically return to the main program.

## COMPUTER OPERATIONS

There are certain operations that are basic to almost any computer. A sound understanding of these basic operations is a necessary prerequisite to examining the specific operations of a particular computer.

### TIMING:

The activities of the central processor are cyclical. The processor fetches an instruction, performs the operations required, fetches the next instruction, and so on. An orderly sequence of events like this requires timing, and the CPU therefore contains a free running oscillator clock which furnishes the reference for all processor actions. The combined fetch and execution of a single instruction is referred to as a *machine cycle*. The portion of a cycle identified with a clearly defined activity is called a phase. And the interval between pulses of the timing oscillator is referred to as a *clock period*. As a general rule, one or more clock periods are necessary to the completion of a phase, and there are several phases in a cycle.

### INSTRUCTION FETCH:

The first phase(s) of any machine cycle will be dedicated to fetching the next instruction. The CPU issues a read operation code and the contents of the program counter are sent to program memory, which responds by returning the next instruction word. The first word of the instruction is placed in the instruction register. If the instruction consists of more than one word, additional cycles are required to fetch each word of the instruction. When the entire instruction is present in the CPU, the program counter is incremented (in preparation for the next instruction fetch) and the instruction is decoded. The operation specified in the instruction will be executed in the remaining phases of the machine cycle. The instruction may call for a memory read or write, an input or output and/or an internal CPU operation, such as a register-to-register transfer or an add-registers operation.

### MEMORY READ:

The instruction fetched may then call for data to be read from data memory into the CPU. The CPU issues a read operation code and sends the proper memory address; memory responds by returning the requested word. The data received is placed in the accumulator (not the instruction register).

### MEMORY WRITE:

A program memory write operation is similar to a read except for the direction of data flow. The CPU issues a write operation code, sends the proper memory address, then sends the data word to be written into the addressed memory location.

### INPUT/OUTPUT:

Input and Output operations are similar to memory read and write operations with the exception that a peripheral I/O port is addressed instead of a memory location. The CPU issues the appropriate input or output command, sends the